ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

INTRODUCTION TO ROBOTICS

PAPER 2 · PRACTICE E · MARKING GUIDELINE

MARKS: 80

Marking guideline for the Academic Central NC(V) Level 3 Introduction to Robotics paper 2 · practice e.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Practical work is not marked strictly on syntax. Give marks for correct logic, correct pins and connections, sensible comments and neat work. Accept other pin choices, layouts and program structures that work.

QUESTION 1

1.1 Entrance IR: VCC, GND ✓ and OUT to a digital pin ✓; exit IR: VCC, GND ✓ and OUT to a digital pin ✓; servo signal to a PWM pin ✓ with 5 V and GND ✓; LCD VCC and GND ✓, SDA to A4 ✓ and SCL to A5 ✓; green LED with 220 Ω to a pin ✓; red LED with 220 Ω to a pin ✓; LED cathodes to GND ✓; neat, labelled, no short circuits ✓✓ (14)

1.2 Start ✓; set up, gate closed, free = 10, show on LCD ✓✓; loop ✓; decision: car at entrance? ✓; decision: free > 0? ✓; Yes – open gate, wait, close, free − 1 ✓✓; No – show FULL ✓; decision: car at exit? ✓ → free + 1 (max 10) ✓; update LCD ✓; decision free = 0 → red on / green off, else green on ✓; loop back ✓ (14)

1.3 Circuit ✓ and flowchart ✓ saved and printed / presented (2)

[30]

QUESTION 2

2.1 Libraries included and objects created ✓✓; pin modes ✓; lcd.init / backlight ✓; servo attached at 0° ✓; initial display "Free bays: 10" ✓; entrance read and tested for LOW ✓; free > 0 check ✓; gate 90°, delay 3 s, 0° ✓✓; free decreased ✓; FULL shown when full ✓; exit sensor increases free ✓ with a maximum of 10 ✓; LCD updated ✓; LEDs set correctly ✓; wait until the car has passed ✓; comments ✓ (18)

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo gate;
const int entryPin = 2, exitPin = 3, greenPin = 6, redPin = 7;
int freeBays = 10;

void showBays() {                        // update the LCD and the LEDs
  lcd.clear();
  lcd.setCursor(0, 0);
  if (freeBays == 0) lcd.print("FULL");
  else { lcd.print("Free bays: "); lcd.print(freeBays); }
  digitalWrite(greenPin, freeBays > 0 ? HIGH : LOW);
  digitalWrite(redPin, freeBays == 0 ? HIGH : LOW);
}

void setup() {
  pinMode(entryPin, INPUT); pinMode(exitPin, INPUT);
  pinMode(greenPin, OUTPUT); pinMode(redPin, OUTPUT);
  gate.attach(9);
  gate.write(0);                         // gate closed
  lcd.init(); lcd.backlight();
  showBays();
}
```

```
void loop() {
  if (digitalRead(entryPin) == LOW) {     // car at the entrance
    if (freeBays > 0) {
      gate.write(90);                     // open the gate
      delay(3000);
      gate.write(0);                      // close the gate
      freeBays--;
    }
    showBays();
    while (digitalRead(entryPin) == LOW) {}   // wait until the car has passed
  }
  if (digitalRead(exitPin) == LOW) {      // car leaving
    if (freeBays < 10) freeBays++;
    showBays();
    while (digitalRead(exitPin) == LOW) {}
  }
}
```

2.2 Saved ✓ and printed / presented ✓ (2)

[20]

QUESTION 3

3.1 const int potPin = A1; ✓ remove / replace the fixed darkLevel ✓; read the potentiometer every loop with analogRead(potPin) ✓✓ into a variable ✓; use it in the comparison (level < darkLevel) ✓✓; printed for testing (optional) ✓; correct logic ✓ (9)

3.2 A brightness variable kept between loops ✓✓; when dark, brightness increases in small steps ✓ up to 255 ✓; when light, it decreases ✓ down to 0 ✓; analogWrite(lightPin, brightness) ✓; step size and delay give about 2 s ✓ (e.g. 5 per step every 40 ms); limits checked ✓ (9)

3.3 const int buttonPin = 2; ✓ pinMode INPUT_PULLUP ✓; bool manual variable ✓; button LOW toggles manual ✓✓ with a debounce delay / wait for release ✓; in manual mode brightness = 255 ✓✓; otherwise normal automatic control ✓ (9)

```
const int ldrPin = A0, potPin = A1, lightPin = 9, buttonPin = 2;
int brightness = 0;
bool manual = false;

void setup() {
  pinMode(lightPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop() {
  if (digitalRead(buttonPin) == LOW) {          // 3.3 toggle manual mode
    manual = !manual;
    delay(300);                                 // debounce
  }
  int level = analogRead(ldrPin);
  int darkLevel = analogRead(potPin);           // 3.1 set by the potentiometer
  bool dark = level < darkLevel;

  if (manual) brightness = 255;                 // 3.3 always fully on
```

```
  else if (dark && brightness < 255) brightness += 5;   // 3.2 fade in
  else if (!dark && brightness > 0) brightness -= 5;    // 3.2 fade out
  brightness = constrain(brightness, 0, 255);

  analogWrite(lightPin, brightness);
  delay(40);                                    // 51 steps x 40 ms ≈ 2 s
}
```

3.4 Final program saved ✓, printed / presented ✓, with comments showing the changes ✓ (3)

[30]

TOTAL: 80

END OF MARKING GUIDELINE